

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/221200457>

Reengineering of Legacy Systems Based on Transformation Using the Object-Oriented Paradigm.

Conference Paper · January 1998

DOI: 10.1109/WCRE.1998.723184 · Source: DBLP

CITATIONS

14

READS

231

3 authors:



[Rosângela Penteado](#)

Federal University of São Carlos

105 PUBLICATIONS 306 CITATIONS

SEE PROFILE



[Paulo Masiero](#)

University of São Paulo

193 PUBLICATIONS 2,134 CITATIONS

SEE PROFILE



[Antonio Francisco Prado](#)

Federal University of São Carlos

102 PUBLICATIONS 476 CITATIONS

SEE PROFILE

Reengineering of Legacy Systems Based on Transformation Using the Object Oriented Paradigm

Rosângela Penteado¹

UFSCar – DC

São Carlos - SP - Brazil

rosangel@dc.ufscar.br

Antonio F. do Prado¹

UFSCar – DC

São Carlos – SP - Brazil

prado@dc.ufscar.br

Paulo C. Masiero²

ICMC - USP

São Carlos – SP - Brazil

masiero@icmc.sc.usp.br

Rosana T. V. Braga

UNIFRAN/ICMC -USP

São Carlos – SP - Brazil

rtvb@icmc.sc.usp.br

Abstract

Legacy systems that were originally developed using the procedure-oriented approach can be reengineered according to the object-oriented paradigm by the process here proposed. Three phases are included in such process: an object oriented reverse engineering phase, in which an object oriented analysis model of the legacy system is produced by Fusion/RE; a segmentation phase, in which the original code is segmented into object oriented methods, keeping the procedural language; and finally a transformation phase, in which the segmented code is transformed to an object oriented language by the Draco-Puc machine. The application of this process to a real mechanic and electric car repair shop system, with 20.000 lines of source code, is described. Samples of the Java code obtained by transforming the original Clipper code are supplied.

1. Introduction

The need of support tools to reengineer systems is widely claimed in the literature. Cremer [2] and Fioravanti [4] are examples. Sneed [15,16,17] describes a tool-supported process applicable to procedural Cobol programs and Gall and others [6, 7, 8] present Corem – a transformation process that uses application domain knowledge and the procedural source code to retrieve an object model. The authors stress that the complete automation of this transformation process is not feasible but that many steps can be supported by tools.

Part of the authors of this paper has been working in the reverse engineering of systems developed in procedural languages, to obtain the corresponding object oriented analysis model [12,13]. Another part has been working in the development of the Draco-Puc machine [14] to do the automatic transformation of software from one language to another.

The recycling of systems towards object orientation has been the concern of several authors [8, 9, 16]. For example, Sneed shows a process for reengineering Cobol programs, originally developed with procedural orientation, to object orientation [16]. His process is done in two phases, the first preserving the original non object-oriented language, in which the system was developed, and the second changing the language to one version of object oriented Cobol. He emphasizes, among others, the following reasons for the application of his process: it makes the best use of the features offered by the object oriented technology, it reduces maintenance costs and it increases the reuse of programs and data in the new architecture of the system. The first phase is very attractive because it permits to obtain several advantages of the object-oriented approach even before doing the second.

Sneed mentions as obstacles to overcome in his process: the difficulty to separate the legacy code into the potential methods, as they are intertwined in the code; the short size of some of these methods and their high number; the difficulty of establishing automatic rules for object definition, because these can only be defined in the context of their use; the difficulty to eliminate redundant code, as it is very difficult to know what is redundant; and the difficulty in eliminating equal and

¹ Financial Support from RHA/CNPq grant 610623/95-8

² Financial Support from CNPq and FAPESP

opposite names, because the legacy system uses the same name for different things.

It is important to notice that Sneed conducts the reengineering without first doing the reverse engineering of the legacy system. In the same Conference where he has shown his work [16], part of the authors of this paper have proposed an approach to this problem, Fusion/RE [12,13], an overall process to do reverse engineering in legacy code based on the Fusion method [3].

The purpose of this paper is to present a reengineering approach that first applies Fusion/RE to a legacy information system, then does its segmentation, preserving the functionality and the programming language, but changing the paradigm: from procedure to object orientation. Afterwards, from the segmented procedural code, object oriented code is automatically generated, using the Draco-Puc transformation system [14].

The technical motivation of the practical application here described is to obtain full expertise in doing the proposed reengineering with the available resources, so as to be able to increase the computer aid to the process with other existing or specially tailored tools. As it has been done, of the three phases that constitute the proposed reengineering process, the first two involve labor intensive activities that compromise the economic gains obtained by improving the system maintainability.

In section 2 the proposed reengineering process is detailed. In section 3 the object-oriented reverse engineering approach is summarized. In section 4 the forward design of the system and the segmentation of the legacy code is described. In section 5 the transformation of the procedural code to an object oriented one is described. In section 6 the validation of this process is discussed. In section 7 work in progress is presented, and in section 8 the conclusions are shown. From section 3 to section 6, a running example is used to illustrate the process.

2. Proposed Reengineering Process

The reengineering process here proposed consists of a reverse engineering phase plus a segmentation phase plus a transformation phase. The first phase is based on Fusion/RE; the second uses an extension to Fusion/RE; and the third uses the Draco-Puc transformation system.

Figure 1 illustrates the proposed reengineering process. In the reverse engineering phase (1) Fusion/RE is applied to obtain, from the legacy code, the Object Oriented Analysis Model. In the subsequent phase (2), the design phase of the Fusion Method is used to guide the code segmentation phase. The resulting segmented code has object orientation features, but continues implemented in its original procedural language. In fact, it is a code programmed with the style of abstract data

types in which the mechanisms of specialization or generalization and aggregation can be simulated. In phase (3) the segmented code is transformed to a real object oriented language, with the support of Draco-Puc. The resulting code can then be validated (4) to ensure that the system functionality remains the same. This process is done with perfect interaction among the three phases, so that the results produced in one phase are used as entries to the subsequent phase.

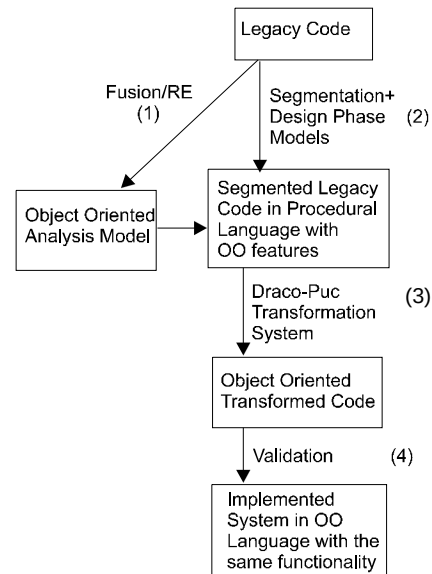


Figure 1. Proposed reengineering process

3. Reverse Engineering

Fusion/RE [12,13] is an overall process for doing reverse engineering that was presented on the III Working Conference on Reverse Engineering. So, here it is only summarized in Figure 2, with the four steps briefly described next. For more details about it, recall [13].

Fusion/RE begins with the legacy system architecture revitalization. First, we must obtain, from the legacy code and existing documentation, a list of all the system procedures and the relationships between them, creating a “call/called by” list. Based on this information and on the system data structures, an object-oriented pseudo-model is created, containing as classes the data structures that are relevant to show the system features. This model is called Current System Analysis Model (CSAM). The procedures of the legacy system usually alter or consult more than one data structure. This is not permissible in object oriented systems, due to the data encapsulation concept. When a procedure interacts with different classes, we say that an anomaly occurs, and it must be

eliminated in the next phase, when an object oriented system analysis model is created.

The creation of the System Analysis Model (SAM) focuses on the application domain and not on the implementation domain. The object, life cycle and operation models are abstracted from those elaborated previously, with additional care as, for example, changing the names to more meaningful ones, doing generalization and specialization of classes, etc. The procedures without anomalies derive methods directly; those with anomalies are split so that each method interacts with one and only one class.

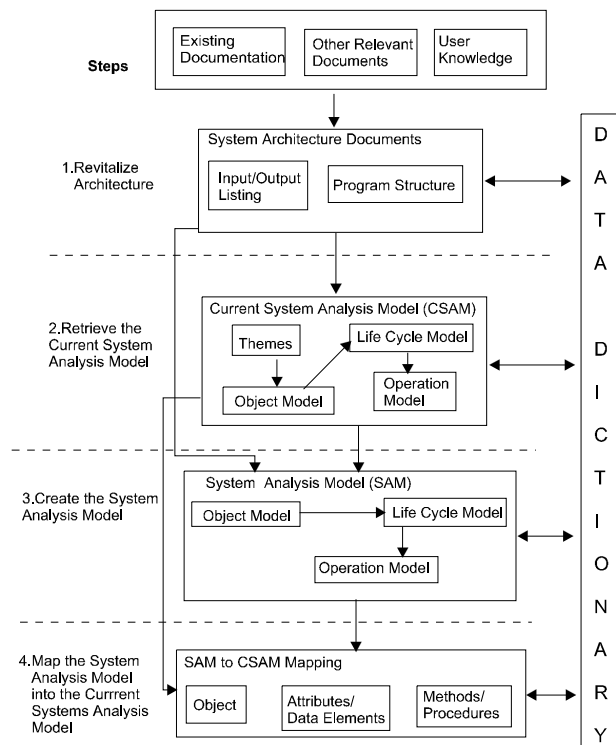


Figure 2. Fusion/RE schema

The mapping from SAM to CSAM describes the relation between them. The classes, attributes, and methods are mapped from SAM to the corresponding elements of CSAM, considering, if possible, inclusions/exclusions. This step is important in future maintenance and reuse of the system, if only reverse engineering is done.

Fusion/RE can be used for systems coded in any procedural language. In this paper, the legacy code was written in Clipper, while in the example of [13] the legacy code was written in C. The example system summarized in Figure 3, has 20,000 lines of source code and its existing documentation consisted only of the list of procedures with a “call/called by” list. Fusion/RE was applied to it, resulting in an Object Model with twenty-six classes, not shown here due to lack of space.

The system deals with the administration of the services done in a mechanic and electric car repair shop, also doing stock control of the parts used in the repairs. The customer goes to the shop to ask for services in his/her vehicle. A customer may have several vehicles. The same vehicle can return to the shop many times and a distinct customer order is issued in each visit. This order contains data about the customer, the vehicle and the services to be done. When the service is completed, the parts used and the work performed are recorded. Sometimes the repair can need parts that do not exist in stock. These are acquired outside the shop and are included in the customer order. To record these parts is important to management, as they are candidates to be stocked. The possible vehicle models are recorded in the system, as there are tables of labor rates prepared according to the vehicle model and the kind of labor.

Figure 3. Partial specification of a mechanic and electric car repair shop system

A small part of such model can be seen in the scheme exhibited on the left side of Figure 4. This figure also shows, on the right side, twenty-four methods (denoted by prefix M1-x), obtained from removing the anomalies of one of the legacy system procedures, called Open_customer_order. This procedure (of which a fragment of the legacy code is shown in Figure 6) is responsible for including a new customer order, when the vehicle comes to the shop to be repaired. It interacts with classes such as: Customer, to fill in the details of the vehicle owner; Vehicle, to identify the vehicle being repaired; Vehicle Model, to define to which model the vehicle belongs to; and Customer Order, to fill in the faults presented, the services to be done, the possible delivery date, etc.

These twenty-four methods were identified by examining both the system execution and the source code. The system execution permits a better understanding of its functionality. Source code examination allows more accurate identification of the methods to be isolated from the procedures in the next phase.

The application of the FusionCase tool [20] both to do the Reverse Engineering and the Forward Design for the Segmentation proved valuable. Other tools, like DBMAIN [21], could be used to ease the initial phases of the reverse engineering if they had support to Clipper.

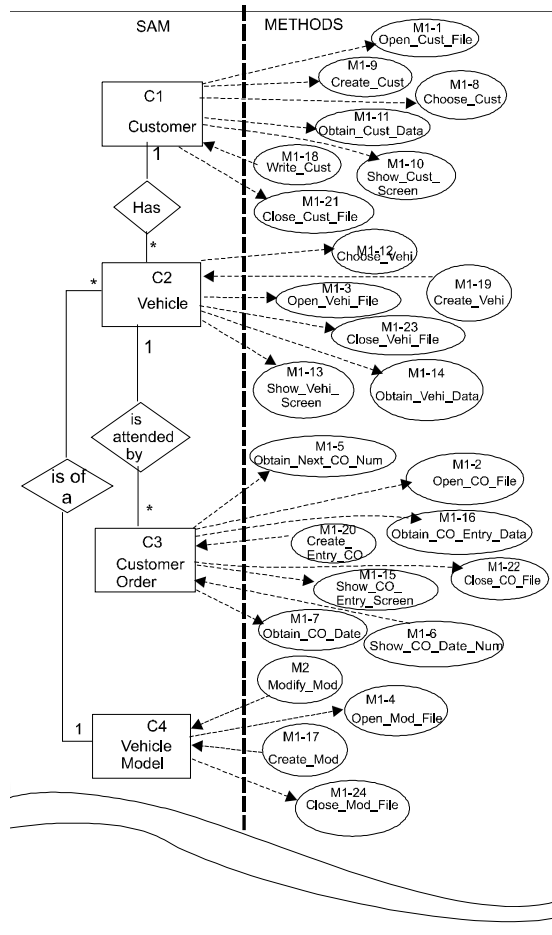


Figure 4. Partial object model and its methods

4. Forward Engineering and Segmentation

In this phase a reengineering for changing the procedure orientation, in which the legacy system has been developed, to object orientation is done. The original programming language is kept.

Two steps are proposed to do this reengineering of the system: “Conduct the Forward System Design” and “Do Program Segmentation”. Figure 5 shows the resulting approach.

The “Conduct the Forward System Design” step considers the reengineering with change of the implementation paradigm, but without changing the functionality. Based on the models obtained in step 3 of Fusion/RE and also on the legacy code, the Object Interaction Graphs, Visibility Graphs, Class Description and Inheritance Graphs are constructed, according to the Fusion Method [3]. These are not shown here due to lack of space.

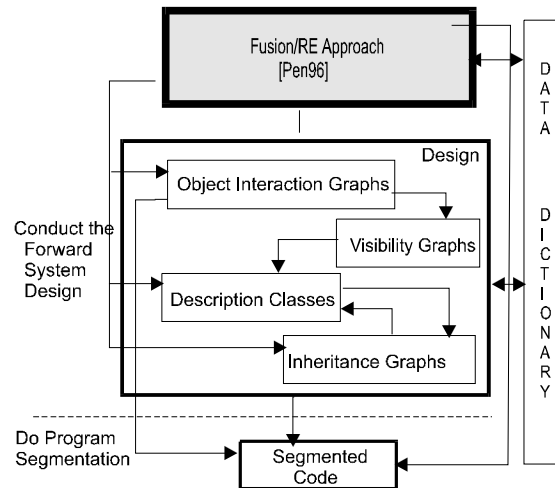


Figure 5. Fusion/RE with the two additional steps

The Object Interaction Graphs are constructed based on the Operation Model. They show the communication between objects, through methods, distinguishing operation controller classes from their collaborator classes. According to the Fusion Method, for each system operation it is necessary to choose a class, among all the classes involved in it, to be its controller class. The other classes are called collaborator classes. A pseudo code is written for the operation showing its logic and the methods calling sequence. It is important to notice that the method of the operation controller class (that implements the operation) interacts with more than one object, but does it by calling the collaborator classes methods, so avoiding anomalies to occur.

The procedures with anomalies can be examined in face of the documentation produced in the last step. The result of this exam will conduct the execution of the next step: “Do Program Segmentation”. In it, the original program can be manually split in methods. The definition of the methods described in section 3, shown in Figure 4, becomes a strong guide for each line of code to be joined with the method that it belongs to. Thus, the difficulties in doing the segmentation, according to Sneed in [16], are attenuated.

The code is examined and comments are inserted establishing a correspondence between this code and the methods defined in the third step. The pseudo code of the method of each operation controller class is reviewed to consider implementation details. After this is done, the parts corresponding to each method are put together. Several alternatives were tested to find the best way for doing the segmentation in Clipper, as it has particular features that make difficult to simulate object orientation. The solution adopted was to create global variables to hold the class attributes that need to be accessed by other classes. These variables are created when the file that

implements the class is opened and updated by class methods. In order to simulate object oriented classes, all the methods belonging to a class were grouped in one physical program file.

As it was said in section 2, the segmentation does not change the non object-oriented programming language. However, after it, the code can be easily translated to an object oriented language as Sneed has done with OO COBOL [16].

Figure 6 shows, on the left column, a piece of the legacy code implemented in Clipper and, on the right column, the corresponding segmented code. As mentioned in section 3, it refers to the legacy system procedure `Open_customer_order`. The number included in parentheses shows, in a rough way; the mapping between the pieces of code on the left column into method calls in right column. We note that the functionality of the legacy system was not altered, but the

segmented code has no anomalies. The names of methods were changed to names with up to 8 characters, because of Clipper limitations. In our work we could corroborate one fact commented by Sneed: the methods are several in quantity but each contains only a few lines of code, enhancing the possibility of reusing them.

The segmentation phase was manually done, taking about ten hours to define and learn the way of doing it and about one-hour to segment, compile and test fifty lines of code. Several problems occurred, but they were solved with little effort. In fact, the legacy code has a uniform structure, so when one problem is solved, the solution is applicable to many similar situations. In other segmentation experiences, done after this work was finished, other problems came out, different from these, as another programmer, with other style and experience wrote the code.

<pre> * Procedure CUS_ORDER_FIRST_PART SELECT 1 USE CUSTOMERS INDEX CUSTCOD, CUSTNAME] (1) SELECT 2 USE CUSTORD INDEX CUSTCO, CUSTORD] (2) SELECT 3 USE CUSTVEHI INDEX CUSTVCOD] (3) . . . DO LOGOTYPE] (4) rec=.f. custcod = " "] (5) vnum = 0 imp = " " ordnum = conumb + 1 entdate = DATE() mv = SPACE(15) tv = SPACE(30) @6,15 SAY "Entry Customer Order-First Part" (6) @7,15 SAY " " @9,5 SAY "Entry Date:" @9,22 GET entdate PICTURE "@E"] (7) @9,48 SAY "C.O. Number:" @9,56 SAY ordnum PICTURE[99999] @10,5 SAY "Customer Code:" @ 10, 24 GET custcod PICTURE [99999]] (8) when cusearch() DO MESSAGE WITH " <99999>- Customer not in file <0> Quit" READ Custcod=val(custcod) . . . IF lastkey()#27 .and. custcod <> 0 custordn=ordnum SELECT CUSTOMER] (9) SET ORDER TO 1 SEEK custcod IF .NOT. FOUND() GO BOTTOM custcod = CUSTCODE + 1 transdat = DATE() APPEND BLANK REPLACE CUSTCOD WITH custcod REPLACE TRANSDATE WITH transdat] (10) . . . </pre>	<pre> * Operation Entry_Customer_Order // Method Open_Cust_File() DO OPENCUST] (1) // Method Open_COrder_File() DO OPENCO] (2) // Method Open_Vehi_File() DO OPENVEHI] (3) . . . DO LOGOTYPE] (4) // Method Obtain_Next_CO_Num() DO OBTNEXCO] (5) ready = .f. // Method Show_CO_Date_Num() DO SHOWCODT] (6) // Method Obtain_CO_Date() DO OBTDATCO] (7) custnew= .f. // Method Choose_Cust() DO CHOOCUST with ready,custnew] (8) . . . if ready // Method Find_Cust() DO FINDCUS] (9) if custnew // Method Create_Cust() DO CREATCUS] (10) endif . . . // Example: method of Customer Order to // obtain the customer order next number Function OBTNEXCO () // Method Obtain_Next_CO_Num] (5) SELECT CUSTORD SET ORDER TO 2 Goto bottom Ordnum = CONUM +1 SET ORDER TO 1 RETURN NIL </pre>
---	--

Figure 6. Clipper legacy code and Clipper segmented code

A future work would be to join all the rules used to segment a program, producing a segmentation algorithm to be followed and possibly automated. This is a hard work, as it depends on the language used and the programming style, experience and tricks used by the programmer.

The segmented code to be submitted to the Draco-Puc machine had to be adjusted in order to pass more smoothly. A sample case is the ordered grouping of certain sets of declarations. If these declarations are not arranged in a pre-established order their recognition will be more difficult. For example, after opening a file, global variables for each field are initialized, making easier for Draco-Puc to specify the class attributes.

5. Code Transformation

In this phase a reengineering for changing the procedure oriented programming language, in which the legacy system has been originally developed, to an object oriented language, is done.

That is, after the segmentation of the source code we do its automatic re-implementation in an object-oriented language. In this case, the segmented Clipper code is used as input to the Draco-Puc machine that transforms it automatically into Java code.

The Draco-Puc machine implements the ideas from the Draco paradigm [10, 14], of software development by domain-oriented transformations. A first prototype of this machine was built by Neighbors [11]. Later, this machine was re-built at PUC-RJ [14], using new languages with more modern hardware and software platforms. Several improvements were introduced in this new version of Draco-Puc [1, 5, 18, 19].

By the strategy proposed by Prado [14], the re-building of software is possible by direct translation from the source code to languages of other domains. A domain, according to Draco-Puc, is composed of three parts:

- A language: defined by a parser that is responsible to analyze the programs of the language and to create the internal representation of Draco-Puc, an Abstract Syntax Tree called DAST.
- A PrettyPrinter or unparser: that formats the DAST, transforming it in text of the domain language. Based on the syntax definition rules of the language grammars, Draco-Puc produces, automatically, the respective domain PrettyPrinters.
- A set of Transformations: one, that maps structures from a domain language to structures of the same domain language, called intra-domain transformation; and the others, that map the applications described in a language of one domain to a language description in another domain, called inter-domain transformation.

In this particular case, to port programs from Clipper to Java, the description of these two domains in Draco-Puc was necessary.

Figure 7 shows a scheme of what is necessary for the construction of the Clipper and Java domains. There are five basic activities that must be done: *Parser Constructing*, *PrettyPrinters Constructing*, *clipperToKB.tfm Constructing*, *clipperToJava.tfm Constructing* and *Libraries Constructing*. Each of these activities is briefly commented, complementing what is shown in figure 7.

In *Parser Constructing* we obtain the context free grammar parsers of the Clipper and Java domains, from the definition of their lexical and syntactical analyzers. This activity is supported by the subsystem, Draco-Puc Parser Generator, named “Pargen”. It is possible to see on the left side of Figure 7 a part of the *Definition of the Clipper* (above) and *Java* (below) *Parsers*.

In *PrettyPrinters Constructing*, Draco-Puc generates automatically the Clipper and Java PrettyPrinters using its “Ppgen” subsystem. Figure 7 shows part of *Clipper* and *Java PrettyPrinters*.

The Draco-Puc machine has a Knowledge Base (KB) to store facts and rules, analog to that of Prolog. In *clipperToKB.tfm Constructing* the “clipperToKB.tfm Transformer” is obtained based on Clipper grammar rules, that recognize procedures, functions, data dependencies and other structures of the language. The “clipperToKB.tfm Transformer” aims to generate facts about the Clipper semantics in the KB by traversing the different structures of the segmented Clipper code.

The KB schema, that oriented the *clipperToKB.tfm Constructing*, was defined based on the SAM and the Clipper grammar rules. The segmented code is supplied to the “clipperToJava.tfm Transformer” to know the methods, variables, types and files of Clipper, in order to give instructions for the creation of the classes and for the encapsulation of these methods and attributes. At the end of this step we have facts as: *variable(name, type)*, *class(name, category)*, *attribute(name, class, type)*, *service(name, class, return)*, and others, which are used in the next step of the transformation from Clipper to Java. In this activity the software engineer uses the Draco-Puc “Tfmgen” subsystem.

In *clipperToJava.tfm Constructing* Draco-Puc generates the inter-domain transformer that does the semantic mapping from Clipper to Java using the facts stored in the Knowledge Base. The semantic of the Clipper commands is preserved in each Java command, because Draco-Puc works with different levels of granularity in the transformation definition, so that each Clipper command is transformed to its corresponding in Java. In this step, the software engineer also uses the “Tfmgen” to generate the “clipperToJava.tfm Transformer”. Figure 7 shows part of the “Definition of ClipperToJava.tfm Transformer”.

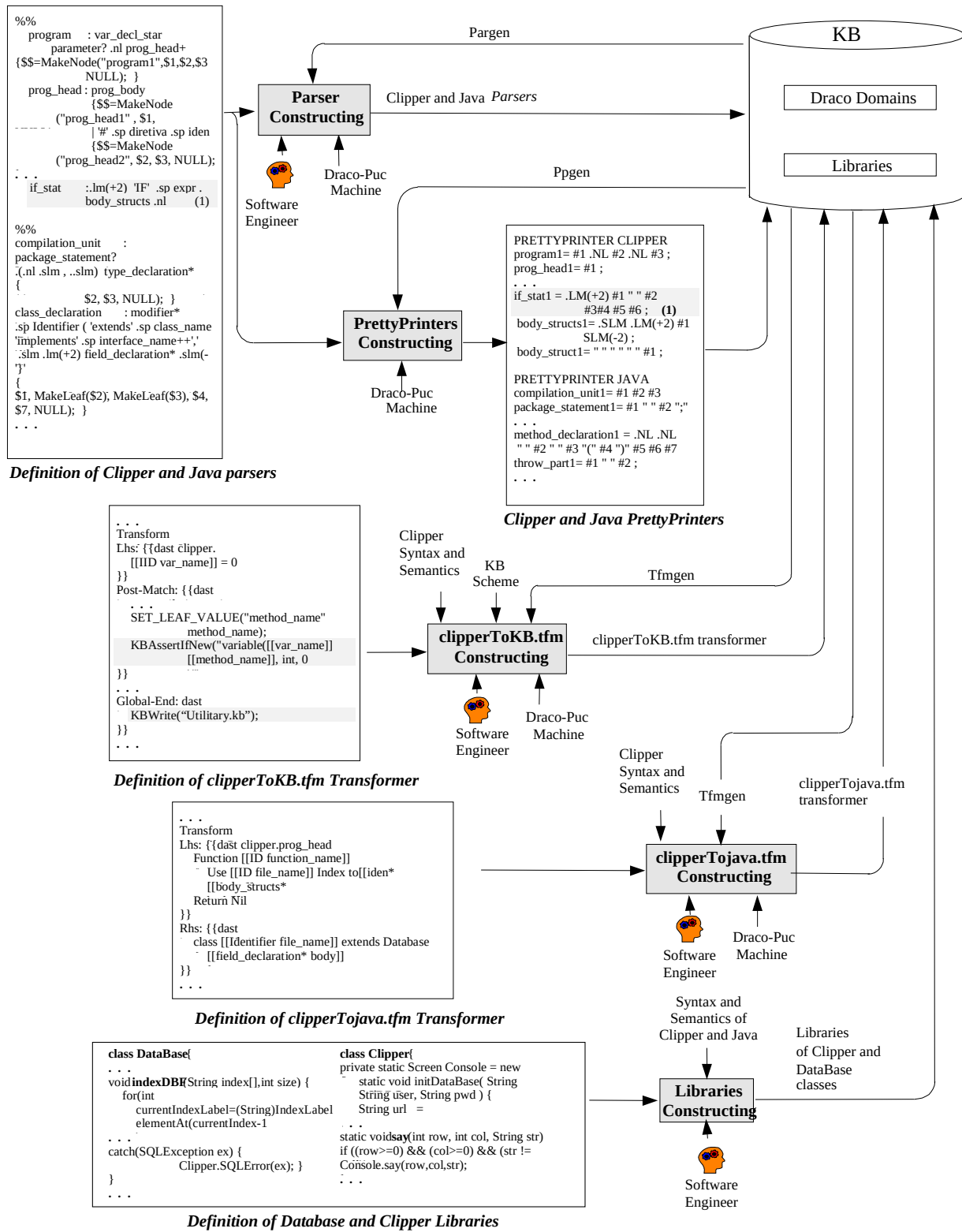


Figure 7. Construction of Clipper and Java domains

Figure 8 shows part of the Java object oriented code corresponding to the segmented Clipper program, (right column of Figure 6), obtained using the DracoPuc machine.

```

Import java.util.*;

Class Cusord extends DataBase{
    public int C0num;
    public Date Entrydate=new Date();
    public String Servaex1=new String(new
    char[60]);
    public char Budget;
    public float Pricetot;
    . . .

public CusOrdv() {
try{
    openDBF("CUSORDER", "NEW");
    String indexex[] = {"CUSTCO", CUSTORD"}
    IndexDBF(indexex,2)
}
catch(SQLException e) {
    Clipper.SQLError(e);
}

public void OP_C01() {
    Cusord. CUSORD = new CusOrd();
    Customer.CUSTOMER= new Customer();
    Customer.CUSTVEHI=new Custvehi();
    Vehitype VEHITYPE =new vehitype()
    Utility.LOGOTYPE()
    int custnum = 0
    String custcod= " ";
    CUSORD.OBTPROX0(custnum); . . .
    if (Clipper.lastkey() !=27&custcod!=0) {
        boolean new=true;
        CUSTOMER.CUSTFOUND(custcod,new);
        if(new){
            CUSTOMER.CUSTNEW()
        }
        else {
Utility.MESSAGE("Verify customer data,
modifying if necessary");
...
        }
    }
}
}

```

Detailed information for the prototype tools, used in this work, can be obtained in [18,19]. The Draco-Puc machine is available to other researchers by contact with draco@dc.ufscar.br.

introduce any errors or undesired functional changes. Figure 9 shows a screen of the Clipper program execution and Figure 10 shows the corresponding

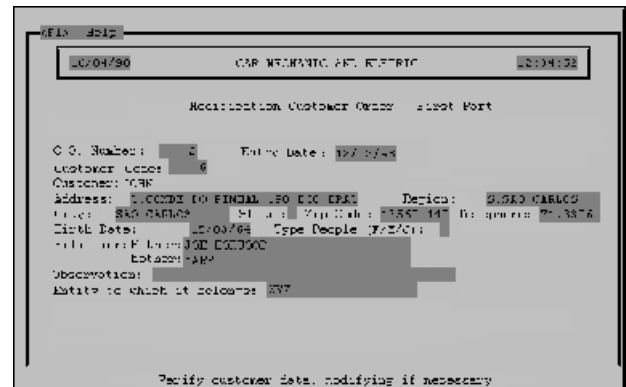
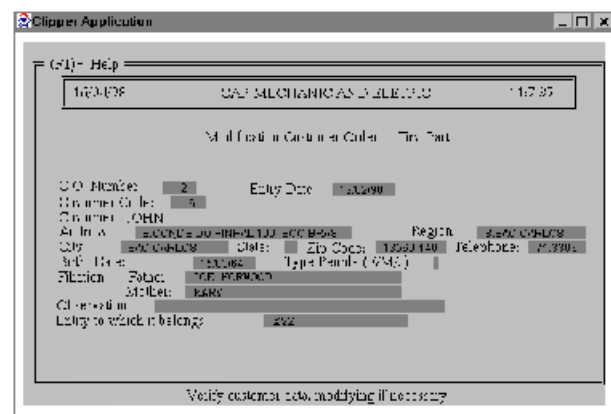


Figure 9. Clipper interface



Test data are prepared and submitted to the original program. The results obtained are saved to be used subsequently in comparisons.

After completion of the transformation, the same test data are submitted to the transformed program and the results obtained are compared with those obtained before. Again, discrepancies or errors have to be analyzed and corrected in an analogous way.

After the transformation of the Clipper program to Java, we can observe that the functionality has not been

altered. The system interface has also remained the same. From the legacy system in Clipper code, with the use of the Draco-Puc machine, it was possible to generate Java code that attends the principles of object orientation and expresses the same semantic of the original code.

It is possible to state that the generated implementation conforms to the original code, because the reengineering, in the reverse engineering and segmentation phases, preserves the functionality of the original program. Besides, the transformations are constructed to preserve the same semantics in the original and target languages grammar rules. The software engineer that constructs the transformations knows both the original and target language.

Although we have used Java as the target language and it is interpreted, we have observed that the code after reengineering has the same performance than the original code. The generated code is organized around the access to a database thus catering for readability and maintainability. It uses objects that encapsulate the methods to access the database; restructures the control flow using object oriented techniques and writes the target source code syntax oriented, using the domain language pretty printer. The code generated preserves the same data and function names of the segmented code.

7. Work in Progress

The system used as example in the paper presented at WCRE 96 [13] is suffering the second phase of the proposed reengineering process, as the reverse engineering phase had already been done. Data is being collected on the amount of effort being done. A part is being manually transformed to C++. Empirical studies will be conducted to quantify the resulting improvement in maintainability.

Else than the practical case here discussed, other systems in Clipper have suffered the two initial phases of the proposed reengineering process. They had even poorer documentation to start with, but the results were equally satisfactory.

8. Conclusions

A process for reengineering programs developed with procedure orientation, obtaining object oriented ones, has been shown. It makes possible to integrate the Fusion/RE approach with the Draco-Puc machine, through the addition of two new steps and new elements in Draco-Puc to deal with other specific domains.

We have shown a particular case as example, using a system implemented in Clipper and transformed to the Java language, illustrating the theory that was presented. The transformation from other procedural languages to other object oriented languages is also possible, with the

support of the Draco-Puc machine. The construction of the respective domains, if they are not yet in Draco-Puc, is necessary. The software engineer does this process, following the details of the Draco-Puc machine.

The re-construction of software, so that it can be executed in new hardware and software platforms, as in the case of Java with database facilities, improves the software without losing the functionality of the legacy code. Moreover, the reverse engineering phase and subsequent forward engineering phase provide good analysis and design documentation, necessary to a better maintainability.

The Draco-Puc machine demands labor intensive activities to set it up for doing the transformation from a certain programming language to another. Once these set up activities are completed, any number of programs can be transformed automatically.

A significant contribution of this paper comes from the possibility that it offers of high level specification reuse that can be computer aided ported to other languages of which the domain have been built in Draco-Puc.

The segmentation process executed manually, only with the support of a text editor, was initially very time consuming, but after some experience, its efficiency improved a great deal. This is due to the repeatable character of the tasks involved in the segmentation. We are convinced that it can be better computer aided, but not fully automated. This improvement would be mainly in the anomaly search facilities.

The Clipper code could be transformed to Java without a previous segmentation but all the advantages of the object oriented paradigm in terms of readability and maintainability would not be obtained. These are assured by the segmentation phase as the reengineering of change of paradigm from procedure oriented to object oriented is done in it.

This experiment was useful because we could validate the use of Fusion/RE in other system implemented in a different procedural language other than the one previously presented [13]. We notice that Fusion/RE was adequate for obtaining the object oriented analysis model of the legacy system. Its integration with the two steps that permit to do the reengineering of the system with change of orientation and without change of functionality is satisfactory. It is left to the analyst to decide if the implementation language is or not to be changed. Even when it is decided not to change the implementation language there is a gain in maintainability.

Acknowledgments: The authors thank Tiemi C. Sakata for her cooperation on the experimentation of the reengineering process. Silvia M. Abrahão, Liliane B.

Nogueira and Maximiliano S. Marques for their cooperation on the transformation phase.

References

- [1] **Bergmann,U.; Prado, A.F., Leite,J.C.S.,** “Object Oriented Software Development Using Draco-Puc Transform System”, in Proceedings of 10th Brazilian Symposium on Software Engineering, São Carlos-SP, 1996 (In Portuguese).
- [2] **Cremer, K.,** “A Tool Supporting the Re-Design of Legacy Applications”, in Proceedings of 2nd Euromicro conference on Software Maintenance and Reengineering - CSMR'98, IEEE. Florence, Italy, 1998, pp. 142-149.
- [3] **Coleman, D. et al ,** Object oriented Development – The Fusion Method. Prentice Hall, 1994.
- [4] **Fioravanti, F., Nesi, P., Perlini, S.,** “A Tool for Process and Product Assessment of C++ Applications”, in Proceedings of 2nd Euromicro conference on Software Maintenance and Reengineering - CSMR'98, IEEE. Florence, Italy, 1998, pp. 89-95.
- [5] **Freitas, F.G.; Leite, J.C.S., Sant’Anna, M.,** “Implementation Aspects of the Syntactical Analyzers Generator for Transform System Support”, in Proceedings of 1st Brazilian Symposium of Programming Languages. Belo Horizonte-MG, 1996. (In Portuguese).
- [6] **Gall, H.; Klösh, R.,** “Capsule Oriented Reverse Engineering for Software Reuse”, in 4th Proceedings of European Software Engineering Conference, ESEC'93, Garmisch-Partenkirchen, Germany, 1993, pp. 418-33.
- [7] **Gall, H.; Klösh, R.,** “Program Transformation to Enhance the Reuse Potential of Procedural Software”, in Proceedings of ACM Symposium on Applied Computing, SAC'94, Phoenix-Arizona. ACM Press, 1994, pp.99-104.
- [8] **Gall, H.; Klösh, R., Mittermeir, R.** -Object oriented Re-Architecting. Proceedings of the 7th International Conference on Software Engineering and Knowledge Engineering. Rockville, Maryland, p. 334-41, 1995.
- [9] **Jacobson, I.; Lindström, F.,** “Re-engineering of Old Systems to an Object oriented Architecture”, in Proceedings of OOPSLA, 1991, pp. 340-350.
- [10] **Leite,J.C.S; Prado,A.F.,** “Design Recovery - A Multi-Paradigm Approach”, in Proceedings of 1st International Workshop on Software Reusability, Dortmund, Germany, 1991.
- [11] **Neighbors, J.M.,** “The Draco Approach to Constructing Software from Reusable Components”, IEEE Transactions on Software Engineering, V.se-10, n. 5, 1984, pp. 564-77.
- [12] **Penteado, R. D.,** A Method of Object Oriented Reverse Engineering. Sc.D Thesis. Instituto de Física de São Carlos-University of São Paulo, Brazil, 251 pp., 1996. (In Portuguese)
- [13] **Penteado, R.D., Germano, F.; Masiero, P.C.,** “An Overall Process Based on Fusion to Reverse Engineer Legacy Code”, in Proceedings of 3rd Working Conference on Reverse Engineering - IEEE. Monterey, California, 1996, pp. 179-188.
- [14] **Prado, A.F. ,** Strategy of Reengineering of Domain Oriented Software. Sc.D Thesis, PUC-RJ, Brazil, 1992.(In Portuguese)
- [15] **Sneed, Harry M.,** “Planning the Reengineering of Legacy Systems”, IEEE Software, v.12, n. 1, 1995, pp. 24-34.
- [16] **Sneed, Harry M.,** “Object oriented COBOL Recycling”, in Proceedings of 3rd Working Conference on Reverse Engineering, IEEE. Monterey, California, 1996, pp. 169-178.
- [17] **Sneed, Harry M.,** “Architecture and Function of a Commercial Software Reengineering Workbench”, in Proceedings of 2nd Euromicro conference on Software Maintenance and Reengineering - CSMR'98, IEEE. Florence, Italy, 1998, pp. 2-10.
- [18] **Sant’ Anna, M.; Leite, J.C.S.P.; Prado, A.F.,** “Draco-Puc: a Workbench for Developing Transformation Based Software Generators”, in Proceedings of ICSE'98, Kyoto, 1998,pp.135-138.
- [19] **Sant’ Anna, M.; Leite, J.C.S.P.; Prado, A.F.,** “A Generative Approach to Componentware”, in. Proceedings of International Workshop on Component Based Software Engineering, Kyoto, 1998, section 3, number 4 (3.4).
- [20] **SoftCase Consulting Ltd,** FusionCASE - User Guide, 1994.
- [21] **Hainaut, J-L - DB-MAIN - Demo Version 2.01,** On-line Help, 1996.